

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Madhav V Nair (1BF24CS159)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Madhav V Nair (**1BM24CS159**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Manjula S
Associate Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	7
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	10
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	13
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	17
6	Implement 0/1 Knapsack problem using dynamic programming.	21
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	24
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	27
9	Implement Fractional Knapsack using Greedy technique.	33
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	36
11	Implement "N-Queens Problem" using Backtracking.	39

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int a[1000][1000], visited[1000], stack[1000];
int top = -1, n;

void topo(int v)
{
    visited[v] = 1;

    for(int i = 0; i < n; i++)
    {
        if(a[v][i] && !visited[i])
            topo(i);
    }

    stack[++top] = v;
}

void reset()
{
    top = -1;
    for(int i = 0; i < n; i++)
        visited[i] = 0;
}

void generateGraph(int n)
{
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            if(i < j)
                a[i][j] = rand() % 2;
            else
```

```

        a[i][j] = 0;
    }
}

int main()
{
    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    }

    reset();

    for(int i = 0; i < n; i++)
    {
        if(!visited[i])
            topo(i);
    }

    printf("\nTopological Ordering:\n");
    while(top != -1)
        printf("%d ", stack[top--]);

    printf("\n\nVertices\tTime(ms)\n");

    for(n = 100; n <= 500; n += 100)
    {
        reset();
        generateGraph(n);

        start = clock();

```

```

    for(int i = 0; i < n; i++)
    {
        if(!visited[i])
            topo(i);
    }

    end = clock();

    time_taken = ((double)(end - start)) * 1000 / CLOCKS_PER_SEC;

    printf("%d\t\t%.2f\n", n, time_taken);
}

return 0;
}

```

OUTPUT:

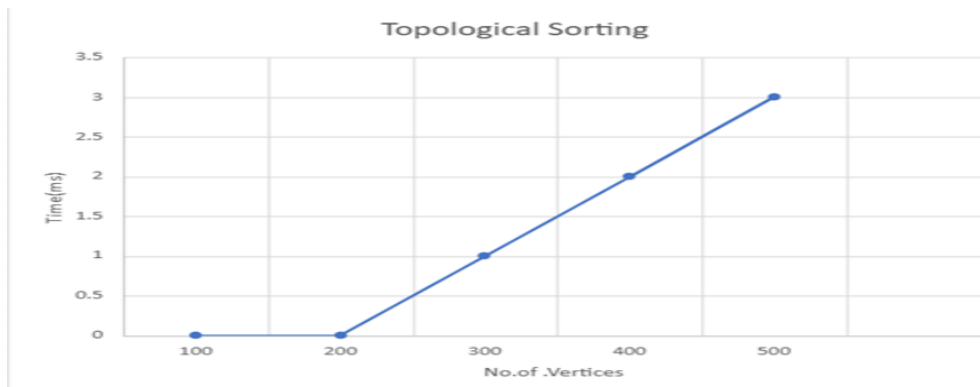
```

Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Topological Ordering:
0 2 1 3

```

GRAPH:



LAB PROGRAM 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

int getMobile(int a[], int dir[], int n)
{
    int mobile = 0, mobile_prev = 0;

    for(int i = 0; i < n; i++)
    {
        if(dir[a[i] - 1] == 0 && i != 0)
        {
            if(a[i] > a[i - 1] && a[i] > mobile_prev)
            {
                mobile = a[i];
                mobile_prev = mobile;
            }
        }

        if(dir[a[i] - 1] == 1 && i != n - 1)
        {
            if(a[i] > a[i + 1] && a[i] > mobile_prev)
            {
                mobile = a[i];
                mobile_prev = mobile;
            }
        }
    }

    return mobile;
}

int searchPos(int a[], int n, int mobile)
{
    for(int i = 0; i < n; i++)
    {
        if(a[i] == mobile)
            return i;
    }

    return -1;
}
```

```

}

void printPermutation(int a[], int n)
{
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

void johnsonTrotter(int n)
{
    int a[n], dir[n];

    for(int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        dir[i] = 0;    // 0 = Left, 1 = Right
    }

    printPermutation(a, n);

    int fact = 1;
    for(int i = 1; i <= n; i++)
        fact *= i;

    for(int i = 1; i < fact; i++)
    {
        int mobile = getMobile(a, dir, n);
        int pos = searchPos(a, n, mobile);

        if(dir[mobile - 1] == 0)
        {
            int temp = a[pos];
            a[pos] = a[pos - 1];
            a[pos - 1] = temp;
            pos--;
        }
        else
        {
            int temp = a[pos];
            a[pos] = a[pos + 1];
            a[pos + 1] = temp;
            pos++;
        }
    }
}

```



```

    }

    for(int j = 0; j < n; j++)
    {
        if(a[j] > mobile)
            dir[a[j] - 1] = 1 - dir[a[j] - 1];
    }

    printPermutation(a, n);
}
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    johnsonTrotter(n);

    return 0;
}

```

OUTPUT:

```

Enter number of elements: 3

1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

```

LAB PROGRAM 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = 0;

    int b[100000];

    while(i <= mid && j <= high)
    {
        if(a[i] < a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }

    while(i <= mid)
        b[k++] = a[i++];

    while(j <= high)
        b[k++] = a[j++];

    for(i = low, k = 0; i <= high; i++, k++)
        a[i] = b[k];
}

void mergesort(int a[], int low, int high)
{

```

```

if(low < high)
{
    int mid = (low + high) / 2;

    mergesort(a, low, mid);
    mergesort(a, mid + 1, high);

    merge(a, low, mid, high);
}
}

int main()
{
    int a[100000], n;
    int values[5] = {100, 500, 1000, 2000, 3000};

    clock_t start, end;
    double time_taken;

    srand(time(NULL));

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergesort(a, 0, n - 1);

    printf("\nSorted elements:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

```

```

printf("\n\nN\tTime(ms)\n");

for(int i = 0; i < 5; i++)
{
    n = values[i];

    for(int j = 0; j < n; j++)
        a[j] = rand() % 10000;

    start = clock();

    mergesort(a, 0, n - 1);

    end = clock();

    time_taken =
        ((double)(end - start)) * 1000 / CLOCKS_PER_SEC;

    printf("%d\t%.2f\n", n, time_taken);
}

return 0;
}

```

OUTPUT:

```

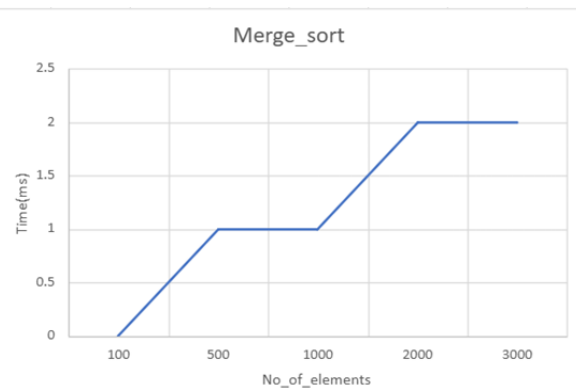
Enter number of elements: 5
Enter elements:
25 80 3 5 90

Sorted elements:
3 5 25 80 90

N      Time(ms)
100    0.00
500    1.00
1000   1.00
2000   2.00
3000   2.00

Process returned 0 (0x0)   execution time : 32.861 s
Press any key to continue.
|

```



LAB PROGRAM 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
int partition(int a[], int low, int high)
```

```
{
```

```
    int pivot = a[low];
```

```
    int i = low + 1;
```

```
    int j = high;
```

```
    int temp;
```

```
    while(1)
```

```
    {
```

```
        while(i <= high && a[i] <= pivot)
```

```
            i++;
```

```
        while(a[j] > pivot)
```

```
            j--;
```

```
        if(i < j)
```

```
        {
```

```
            temp = a[i];
```

```
            a[i] = a[j];
```

```
            a[j] = temp;
```

```
        }
```

```
        else
```

```
        {
```

```
            break;
```

```
        }
```

```
    }
```

```

    temp = a[low];
    a[low] = a[j];
    a[j] = temp;

    return j;
}

void quicksort(int a[], int low, int high)
{
    if(low < high)
    {
        int p = partition(a, low, high);

        quicksort(a, low, p - 1);
        quicksort(a, p + 1, high);
    }
}

int main()
{
    int a[100000], b[100000], n;
    int values[5] = {100, 500, 1000, 2000, 3000};

    clock_t start, end;
    double time_taken;

    srand(time(NULL));

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

```

```

for(int i = 0; i < n; i++)
    scanf("%d", &a[i]);

quicksort(a, 0, n - 1);

printf("\nSorted elements:\n");

for(int i = 0; i < n; i++)
    printf("%d ", a[i]);

printf("\n\nN\tTime(ms)\n");

for(int i = 0; i < 5; i++)
{
    n = values[i];

    for(int j = 0; j < n; j++)
        a[j] = rand() % 10000;

    start = clock();

    for(int k = 0; k < 500; k++)
    {
        for(int j = 0; j < n; j++)
            b[j] = a[j];

        quicksort(b, 0, n - 1);
    }

    end = clock();

    time_taken =
        ((double)(end - start)) * 1000 / CLOCKS_PER_SEC;

```

```

    printf("%d\t%.2f\n", n, time_taken);
}

return 0;
}

```

OUTPUT:

```

Enter number of elements: 10
Enter elements:
90 80 55 45 12 22 35 100 65 75

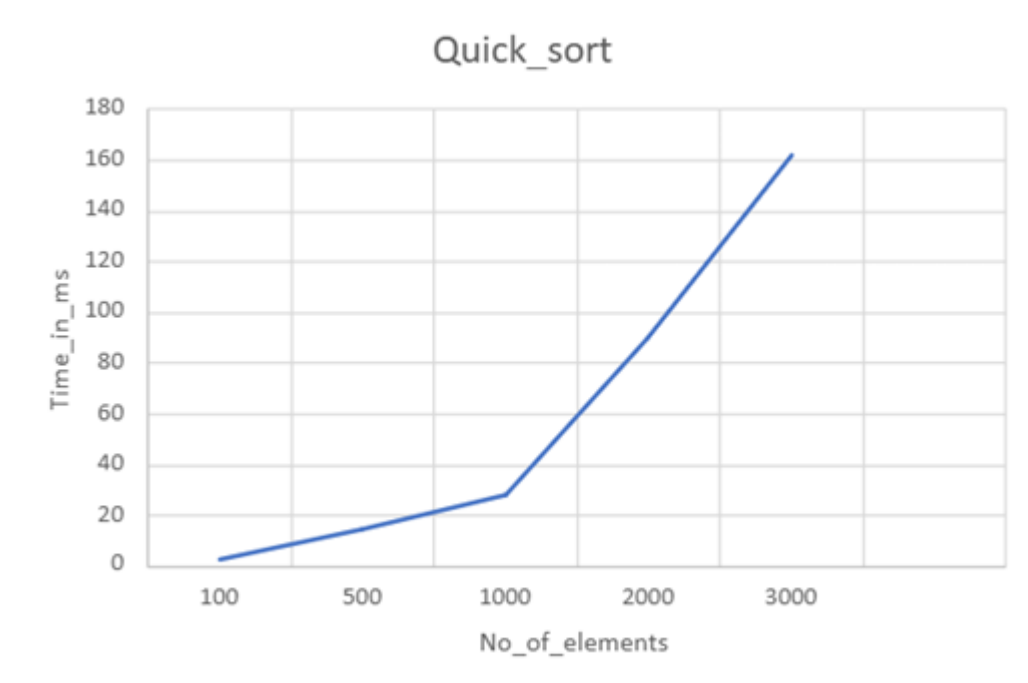
Sorted elements:
12 22 35 45 55 65 75 80 90 100

N      Time(ms)
100    2.00
500    14.00
1000   28.00
2000   90.00
3000   162.00

Process returned 0 (0x0)   execution time : 110.018 s
Press any key to continue.
|

```

GRAPH :



LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void heapify(int a[], int n, int i)
```

```
{
```

```
    int largest = i;
```

```
    int l = 2 * i + 1;
```

```
    int r = 2 * i + 2;
```

```
    int temp;
```

```
    if(l < n && a[l] > a[largest])
```

```
        largest = l;
```

```
    if(r < n && a[r] > a[largest])
```

```
        largest = r;
```

```
    if(largest != i)
```

```
    {
```

```
        temp = a[i];
```

```
        a[i] = a[largest];
```

```
        a[largest] = temp;
```

```
        heapify(a, n, largest);
```

```
    }
```

```
}
```

```
void heapsort(int a[], int n)
```

```
{
```

```
    for(int i = n/2 - 1; i >= 0; i--)
```

```
        heapify(a, n, i);
```

```

    for(int i = n - 1; i > 0; i--)
    {
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main()
{
    int a[100000], b[100000], n;
    int values[5] = {100, 500, 1000, 2000, 3000};

    clock_t start, end;
    double time_taken;

    srand(time(NULL));

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    heapsort(a, n);

    printf("\nSorted elements:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

```

```

printf("\n\nN\tTime(ms)\n");

for(int i = 0; i < 5; i++)
{
    n = values[i];

    for(int j = 0; j < n; j++)
        a[j] = rand() % 10000;

    start = clock();

    for(int k = 0; k < 500; k++)
    {
        for(int j = 0; j < n; j++)
            b[j] = a[j];

        heapsort(b, n);
    }

    end = clock();

    time_taken =
        ((double)(end - start)) * 1000 / CLOCKS_PER_SEC;

    printf("%d\t%.2f\n", n, time_taken);
}

return 0;
}

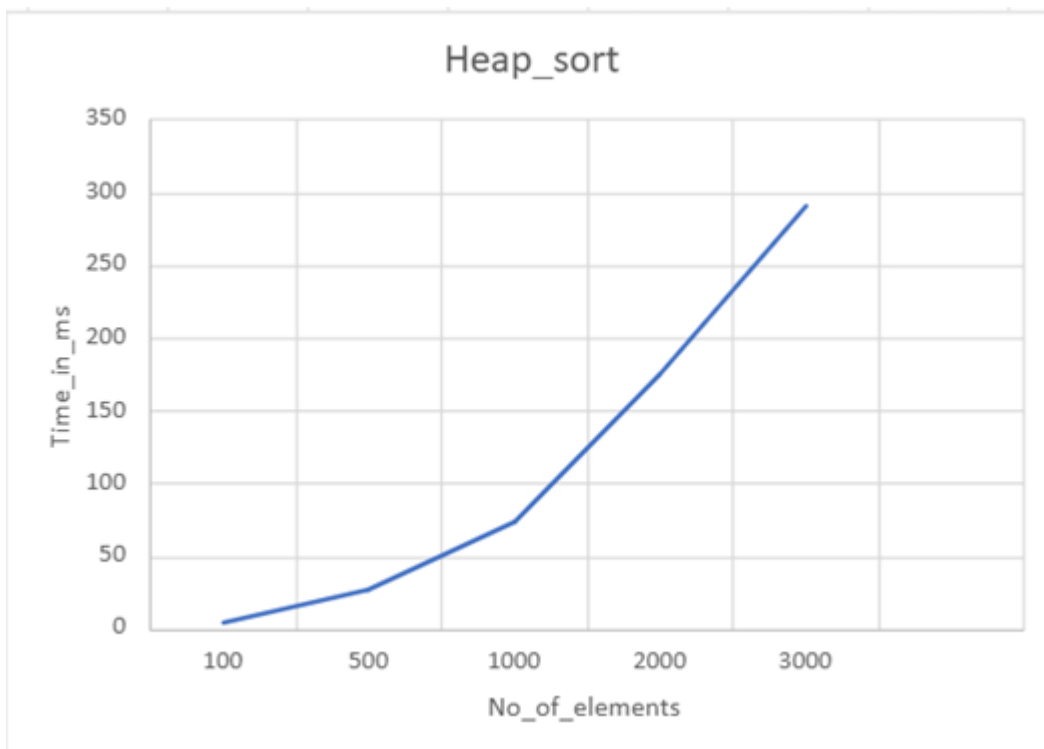
```

OUTPUT:

```
Enter number of elements: 8
Enter elements:
45 12 89 33 7 64 21 10

Sorted elements:
7 10 12 21 33 45 64 89
```

GRAPH:



LAB PROGRAM 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>
```

```
int max(int a, int b)
```

```
{  
    return (a > b) ? a : b;  
}
```

```
int knapsack(int W, int wt[], int val[], int n)
```

```
{  
    int i, w;  
  
    int K[n + 1][W + 1];  
  
    for(i = 0; i <= n; i++)  
    {  
        for(w = 0; w <= W; w++)  
        {  
            if(i == 0 || w == 0)  
            {  
                K[i][w] = 0;  
            }  
            else if(wt[i - 1] <= w)  
            {  
                K[i][w] = max(  
                    val[i - 1] + K[i - 1][w - wt[i - 1]],  
                    K[i - 1][w]  
                );  
            }  
            else  
            {  
                K[i][w] = K[i - 1][w];  
            }  
        }  
    }  
}
```

```

    }
}

return K[n][W];
}

int main()
{
    int n, W, i;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int wt[n], val[n];

    printf("Enter weights of items: ");
    for(i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    printf("Enter values of items: ");
    for(i = 0; i < n; i++)
        scanf("%d", &val[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    printf("Maximum value = %d\n",
        knapsack(W, wt, val, n));

    return 0;
}

```

OUTPUT:

```
Enter number of items: 4
Enter weights of items: 3 4 6 2
Enter values of items: 7 6 9 3
Enter knapsack capacity: 10
Maximum value = 16
```

LAB PROGRAM 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>

#define INF 999

int main()
{
    int n, i, j, k;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int dist[n][n], graph[n][n];

    printf("Enter adjacency matrix:\n");

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);

            if(i != j && graph[i][j] == 0)
                dist[i][j] = INF;
            else
                dist[i][j] = graph[i][j];
        }
    }

    printf("\nGraph with Weights:\n");

    for(i = 0; i < n; i++)
    {
```



```

    for(j = 0; j < n; j++)
        printf("%d\t", graph[i][j]);

    printf("\n");
}

for(k = 0; k < n; k++)
{
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            if(dist[i][k] != INF &&
               dist[k][j] != INF &&
               dist[i][k] + dist[k][j] < dist[i][j])
            {
                dist[i][j] =
                    dist[i][k] + dist[k][j];
            }
        }
    }
}

printf("\nAll Pair Shortest Path Matrix:\n");

for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        if(dist[i][j] == INF)
            printf("INF\t");
        else
            printf("%d\t", dist[i][j]);
    }
}

```

```
        printf("\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 3 0 7
8 0 2 0
5 0 0 1
2 0 0 0

Graph with Weights:
0      3      0      7
8      0      2      0
5      0      0      1
2      0      0      0

All Pair Shortest Path Matrix:
0      3      5      6
5      0      2      3
3      6      0      1
2      5      7      0
```

LAB PROGRAM 8(A):

Find Minimum cost spanning tree of a given undirected graph using Prims Algorithm

```
#include <stdio.h>
```

```
#define INF 999
```

```
int main()
```

```
{
```

```
    int n, i, j;
```

```
    int min, u = 0, v = 0;
```

```
    int total = 0, edges = 0;
```

```
    printf("Enter number of vertices: ");
```

```
    scanf("%d", &n);
```

```
    int cost[n][n];
```

```
    int visited[n];
```

```
    printf("Enter cost adjacency matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        visited[i] = 0;
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            scanf("%d", &cost[i][j]);
```

```
            if(i != j && cost[i][j] == 0)
```

```
                cost[i][j] = INF;
```

```
        }
```

```

}

visited[0] = 1;

printf("\nEdges in Minimum Cost Spanning Tree:\n");

while(edges < n - 1)
{
    min = INF;

    for(i = 0; i < n; i++)
    {
        if(visited[i])
        {
            for(j = 0; j < n; j++)
            {
                if(!visited[j] &&
                    cost[i][j] < min)
                {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }
    }

    printf("%d - %d : %d\n", u, v, min);

    total += min;
    visited[v] = 1;
}

```

```
        edges++;  
    }  
  
    printf("\nMinimum Cost = %d\n", total);  
  
    return 0;  
}
```

OUTPUT:

```
Enter number of vertices: 5  
Enter cost adjacency matrix:  
0 2 0 6 0  
2 0 3 8 5  
0 3 0 0 7  
6 8 0 0 9  
0 5 7 9 0  
  
Edges in Minimum Cost Spanning Tree:  
0 - 1 : 2  
1 - 2 : 3  
1 - 4 : 5  
0 - 3 : 6  
  
Minimum Cost = 16
```

LAB PROGRAM 8(b) :

Find Minimum cost spanning tree of a given undirected graph using Krushkal's Algorithm

```
#include <stdio.h>
```

```
#define INF 999
```

```
int parent[20];
```

```
int find(int i)
{
    while(parent[i])
        i = parent[i];

    return i;
}
```

```
int uni(int i, int j)
{
    if(i != j)
    {
        parent[j] = i;
        return 1;
    }

    return 0;
}
```

```
int main()
{
    int n, i, j;
    int min, a, b, u, v;
    int cost[20][20];
    int edges = 0;
    int total = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        parent[i] = 0;

    printf("Enter cost adjacency matrix:\n");
```

```

for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        scanf("%d", &cost[i][j]);

        if(i != j && cost[i][j] == 0)
            cost[i][j] = INF;
    }
}

printf("\nEdges in Minimum Cost Spanning Tree:\n");

while(edges < n - 1)
{
    min = INF;

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            if(cost[i][j] < min)
            {
                min = cost[i][j];
                a = u = i;
                b = v = j;
            }
        }
    }

    u = find(u);
    v = find(v);

    if(uni(u, v))
    {
        printf("%d - %d : %d\n", a, b, min);
        total += min;
        edges++;
    }

    cost[a][b] = cost[b][a] = INF;
}

printf("\nMinimum Cost = %d\n", total);

```

```
    return 0;  
}
```

OUTPUT:

```
Enter number of vertices: 4  
Enter cost adjacency matrix:  
999 2 999 6  
2 999 3 8  
999 3 999 5  
6 8 5 999  
  
Edges in Minimum Cost Spanning Tree:  
0 - 1 : 2  
1 - 2 : 3  
2 - 3 : 5  
  
Minimum Cost = 10
```


LAB PROGRAM 9:

Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>

struct Item
{
    int weight;
    int profit;
    float ratio;
};

int main()
{
    int n, i, j, capacity;
    float totalProfit = 0;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item item[n];

    printf("Enter weight and profit of each item:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d %d",
            &item[i].weight,
            &item[i].profit);

        if(item[i].weight == 0)
        {
            printf("Weight cannot be zero.\n");
            return 0;
        }
    }
```

```

    item[i].ratio =
        (float)item[i].profit /
        item[i].weight;
}

printf("Enter knapsack capacity: ");
scanf("%d", &capacity);

for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(item[i].ratio < item[j].ratio)
        {
            struct Item temp = item[i];
            item[i] = item[j];
            item[j] = temp;
        }
    }
}

printf("\nSelected Items:\n");

for(i = 0; i < n; i++)
{
    if(capacity >= item[i].weight)
    {
        capacity -= item[i].weight;

        totalProfit += item[i].profit;

        printf("Item %d fully selected\n",
            i + 1);
    }
}

```

```

else
{
    totalProfit +=
        item[i].ratio * capacity;

    printf("Item %d partially selected\n",
        i + 1);

    break;
}
}

printf("\nMaximum Profit = %.2f\n",
    totalProfit);

return 0;
}

```

OUTPUT:

```

Enter number of items: 7
Enter weight and profit of each item:
1 5
3 10
5 15
4 7
1 8
3 9
2 4
Enter knapsack capacity: 15

Selected Items:
Item 1 fully selected
Item 2 fully selected
Item 3 fully selected
Item 4 fully selected
Item 5 fully selected
Item 6 fully selected
Item 7 partially selected

Maximum Profit = 51.00

```

LAB PROGRAM 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
```

```
#define INF 999
```

```
int main()
```

```
{
```

```
    int n, i, j;
```

```
    int source;
```

```
    int count, min, next;
```

```
    printf("Enter number of vertices: ");
```

```
    scanf("%d", &n);
```

```
    int cost[n][n];
```

```
    int dist[n];
```

```
    int visited[n];
```

```
    printf("Enter cost adjacency matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            scanf("%d", &cost[i][j]);
```

```
            if(i != j && cost[i][j] == 0)
```

```
                cost[i][j] = INF;
```

```
        }
```

```
    }
```

```

printf("Enter source vertex: ");
scanf("%d", &source);

for(i = 0; i < n; i++)
{
    dist[i] = cost[source][i];
    visited[i] = 0;
}

dist[source] = 0;
visited[source] = 1;

count = 1;

while(count < n - 1)
{
    min = INF;
    next = -1;

    for(i = 0; i < n; i++)
    {
        if(!visited[i] && dist[i] < min)
        {
            min = dist[i];
            next = i;
        }
    }

    if(next == -1)
        break;

    visited[next] = 1;

    for(i = 0; i < n; i++)
    {

```

```

        if(!visited[i] &&
           min + cost[next][i] < dist[i])
        {
            dist[i] =
                min + cost[next][i];
        }
    }

    count++;
}

printf("\nShortest distances from vertex %d:\n",
       source);

for(i = 0; i < n; i++)
{
    if(dist[i] == INF)
        printf("%d -> %d = INF\n",
               source, i);
    else
        printf("%d -> %d = %d\n",
               source, i, dist[i]);
}

return 0;
}

```

OUTPUT:

```

Enter number of vertices: 4
Enter cost adjacency matrix:
0 10 20 30
1 0 5 15
1 20 0 30
2 1 6 0
Enter source vertex: 0

Shortest distances from vertex 0:
0 -> 0 = 0
0 -> 1 = 10
0 -> 2 = 15
0 -> 3 = 25

```

LAB PROGRAM 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int x[20], n;
```

```
int solutions = 0;
```

```
int place(int k, int i)
```

```
{
```

```
    int j;
```

```
    for(j = 1; j < k; j++)
```

```
    {
```

```
        if(x[j] == i ||
```

```
           abs(x[j] - i) == abs(j - k))
```

```
            return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
void nQueens(int k)
```

```
{
```

```
    int i, j;
```

```

for(i = 1; i <= n; i++)
{
    if(place(k, i))
    {
        x[k] = i;

        if(k == n)
        {
            solutions++;

            printf("\nSolution %d:\n",
                solutions);

            for(j = 1; j <= n; j++)
            {
                for(i = 1; i <= n; i++)
                {
                    if(x[j] == i)
                        printf("Q ");
                    else
                        printf(". ");
                }

                printf("\n");
            }
        }
        else
        {
            nQueens(k + 1);
        }
    }
}
}

```



```

int main()
{
    printf("Enter number of queens: ");
    scanf("%d", &n);

    nQueens(1);

    if(solutions == 0)
        printf("\nNo solution exists.\n");

    return 0;
}

```

OUTPUT:

```

Enter number of queens: 4

Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .

```

LEETCODE PROBLEMS :

1. Leetcode 190:

Reverse Bits

```
#include <stdint.h>
```

```
uint32_t reverseBits(int n)
{
    uint32_t num = (uint32_t)n;

    uint32_t result = 0;
    int i;

    for(i = 0; i < 32; i++)
    {
        result <<= 1;
        result |= (num & 1);
        num >>= 1;
    }

    return result;
}
```

Output:

The screenshot shows the output of a C++ program for the 'Reverse Bits' problem. At the top, there are two tabs: 'Case 1' and 'Case 2', both with green checkmarks. Below the tabs, the 'Input' section shows 'n =' followed by the value '43261596'. The 'Output' section shows the result '964176192'. The 'Expected' section also shows '964176192', indicating that the program's output matches the expected result.

Section	Value
Input	n = 43261596
Output	964176192
Expected	964176192

Leetcode 316:

```
#include <string.h>
#include <stdbool.h>
#include <stdlib.h>

// Function required by LeetCode
char* removeDuplicateLetters(char* s)
{
    int n = strlen(s);

    int count[26] = {0};
    bool used[26] = {false};

    // Count occurrences
    for(int i = 0; i < n; i++)
    {
        count[s[i] - 'a']++;
    }

    // Stack to build result
    char *stack = (char *)malloc((n + 1) * sizeof(char));

    int top = -1;

    for(int i = 0; i < n; i++)
    {
        char c = s[i];
        int idx = c - 'a';

        count[idx]--;

        if(used[idx])
            continue;
```

```

while(top >= 0 &&
    stack[top] > c &&
    count[stack[top] - 'a'] > 0)
{
    used[stack[top] - 'a'] = false;
    top--;
}

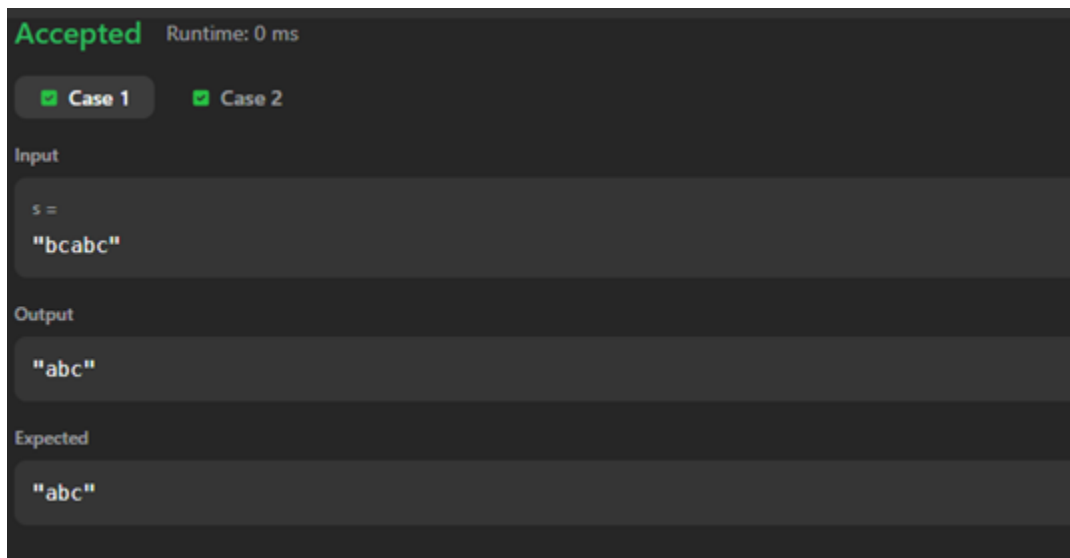
stack[++top] = c;
used[idx] = true;
}

stack[top + 1] = '\0';

return stack;
}

```

OUTPUT:



Leetcode 1221: Split a string in Balanced String

```
class Solution {  
    public int balancedStringSplit(String s) {  
        int balance = 0;  
        int count = 0;  
  
        for(char ch : s.toCharArray()) {  
            if(ch == 'L')  
                balance++;  
            else  
                balance--;  
  
            if(balance == 0)  
                count++;  
        }  
  
        return count;  
    }  
}
```

OUTPUT:

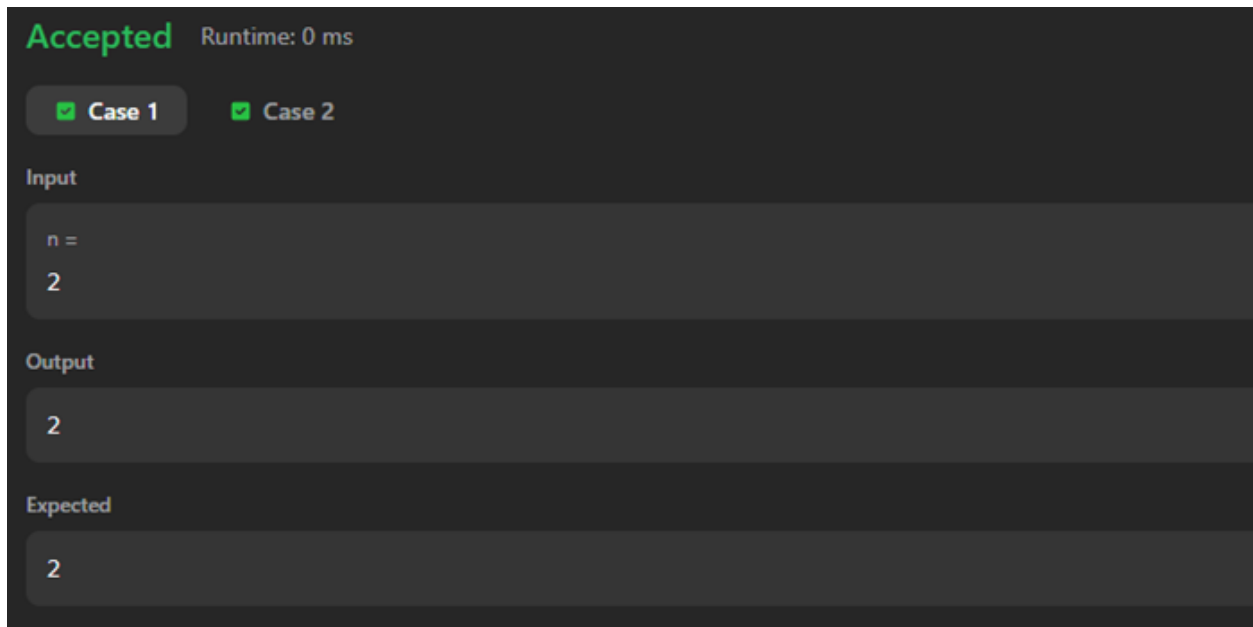


The screenshot displays a LeetCode submission interface. At the top, it shows "Accepted" in green text and "Runtime: 0 ms". Below this, there are three tabs labeled "Case 1", "Case 2", and "Case 3", all of which are checked with green boxes. The "Input" section shows "s =" followed by the string "RLRRLLRLRL". The "Output" section shows the number "4". The "Expected" section also shows the number "4".

Leetcode 70: Climbing Stairs

```
class Solution {  
    public int climbStairs(int n) {  
        if (n <= 2)  
            return n;  
  
        int first = 1, second = 2;  
  
        for (int i = 3; i <= n; i++) {  
            int third = first + second;  
            first = second;  
            second = third;  
        }  
  
        return second;  
    }  
}
```

OUTPUT:



The screenshot displays a dark-themed interface for a LeetCode submission. At the top, the word "Accepted" is written in green, followed by "Runtime: 0 ms". Below this, there are two buttons: "Case 1" and "Case 2", both with green checkmarks indicating they passed. The "Input" section shows "n =" followed by the value "2". The "Output" section shows the value "2". The "Expected" section also shows the value "2".

Leetcode 494 : Target Sum

```
#include <stdio.h>

int solve(int nums[], int size, int target, int index, int sum)
{
    if(index == size)
    {
        return (sum == target) ? 1 : 0;
    }

    int add = solve(nums, size, target,
                    index + 1,
                    sum + nums[index]);

    int subtract = solve(nums, size, target,
                        index + 1,
                        sum - nums[index]);

    return add + subtract;
}

int findTargetSumWays(int nums[], int size, int target)
{
    return solve(nums, size, target, 0, 0);
}

int main()
{
    int n, target;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int nums[n];
```

```
printf("Enter elements: ");
for(int i = 0; i < n; i++)
    scanf("%d", &nums[i]);

printf("Enter target: ");
scanf("%d", &target);

int ways = findTargetSumWays(nums, n, target);

printf("Number of ways = %d\n", ways);

return 0;
}
```

OUTPUT:



Leetcode 207: Course Schedule

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int canFinish(int numCourses, int prerequisites[][2], int prereqSize)
{
    int graph[MAX][MAX] = {0};
    int indegree[MAX] = {0};

    // Build graph
    for(int i = 0; i < prereqSize; i++)
    {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

        graph[prereq][course] = 1;
        indegree[course]++;
    }

    int queue[MAX];
    int front = 0, rear = -1;

    // Add vertices with indegree 0
    for(int i = 0; i < numCourses; i++)
    {
        if(indegree[i] == 0)
            queue[++rear] = i;
    }

    int count = 0;

    while(front <= rear)
```

```

{
    int node = queue[front++];
    count++;

    for(int i = 0; i < numCourses; i++)
    {
        if(graph[node][i])
        {
            indegree[i]--;

            if(indegree[i] == 0)
                queue[++rear] = i;
        }
    }
}

return (count == numCourses);
}

int main()
{
    int numCourses, prereqSize;

    printf("Enter number of courses: ");
    scanf("%d", &numCourses);

    printf("Enter number of prerequisite pairs: ");
    scanf("%d", &prereqSize);

    int prerequisites[MAX][2];

    printf("Enter prerequisite pairs (course prerequisite):\n");

    for(int i = 0; i < prereqSize; i++)
    {

```

```

scanf("%d %d",
      &prerequisites[i][0],
      &prerequisites[i][1]);
}

if(canFinish(numCourses,
            prerequisites,
            prereqSize))
{
    printf("All courses can be completed.\n");
}
else
{
    printf("Cycle detected. Courses cannot be completed.\n");
}

return 0;
}

```

OUTPUT:

Accepted
Runtime: 0 ms

☒ Case 1
☒ Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

